OSD使用说明

1. OSD功能介绍

OSD（On-Screen Display，屏幕叠加显示）是一种在视频帧或显示缓冲区上**叠加图形或文本信息**的技术，常用于在不改变原始画面语义的前提下，显示辅助信息。OSD 库的软件开发库提供的API接口，支持设备管理，图层管理，DMA缓冲区管理、四边形图形操作，位图操作等功能。位图支持特殊格式ssbmp，转换工具使用方法见文档最后位图格式转换部分。

2. 一般流程示例

以SDK中自带的人脸识别ssne_ai_demo使用到的OSD功能为例，具体可以自行开发DIY，一般流程仅作参考。

引用头文件（OSD相关）

开发主要用到的头文件是osd_lib_api.h和osd_lib_types.h，其余OSD相关的头文件与实际开发关系不大。

```
#include "osd_lib_api.h"        //必须引用，包含了所有公共API接口的函数声明
#include "osd_lib_types.h"       //包含所有OSD数据类型定义（结构体、枚举、类型别名
```

OSD功能依赖库列表

库文件路径链接方式的cmake写法可以参考人脸识别ssne_ai_demo中的Paths.cmake

依赖库列表：

libosd.so

libcmabuffer.so

libsszlog.so

libzlog.so

libemb.so

初始化OSD设备资源

打开OSD设备 →初始化OSD设备 →分配DMA缓冲区 →创建图层 →缓冲区绑定到图层

API调用：

打开OSD设备：osd_open_device
初始化OSD设备：osd_init_device
分配DMA缓冲区：osd_alloc_buffer

创建图层：osd_create_layer
缓冲区绑定到图层：osd_set_layer_buffer

绘制GRAPHIC

1.四边形绘制（TYPE_）

将四边形添加到任意图层 →刷新所有图层的四边形数据

API调用：

将四边形添加到任意图层：osd_add_quad_rangle
刷新所有图层的四边形数据：osd_flush_quad_rangle

将四边形添加到指定图层 →刷新指定图层的四边形数据

API调用：

将四边形添加到指定图层：osd_add_quad_rangle_layer
刷新指定图层的四边形数据：osd_flush_quad_rangle_layer

2.位图绘制（TYPE_IMAGE）

将位图添加到任意图层 →刷新所有图层的位图数据

API调用：

将位图添加到任意图层：osd_add_texture
刷新所有图层的位图数据：osd_flush_texture

将位图添加到指定图层 →刷新指定图层的位图数据

API调用：

将位图添加到指定图层：osd_add_texture_layer
刷新指定图层的位图数据：osd_flush_texture_layer

释放OSD设备资源

销毁所有图层 →释放DMA资源 →关闭OSD设备

API调用：

销毁所有图层：osd_destroy_layer
释放DMA资源：osd_delete_buffer
关闭OSD设备：osd_init_device

3. API使用说明

3.1 设备管理

osd_open_device

函数原型

```
handle_t osd_open_device(void);
```

功能描述

打开 OSD 设备，创建设备句柄。**必须在使用其他 API 之前调用。**

返回值

- 成功：**返回有效设备句柄handle**（非 0）
- 失败：返回 INVALID_HANDLE (0)

使用示例

```
m_osd_handle = osd_open_device();      // 打开OSD设备，获取设备句柄 m_osd_hanc
```

---

osd_close_device

函数原型

```
int osd_close_device(handle_t handle);
```

功能描述

关闭 OSD 设备，释放设备句柄和相关资源。

参数

- handle：设备句柄，由 osd_open_device 返回

返回值

- 成功：0
- 失败：-1（通常因为句柄无效）

注意事项

- 关闭前建议先清理所有图层和缓冲区
- 关闭后句柄无效，不可再使用
- 建议程序退出前调用

---

osd_init_device

函数原型

```
int osd_init_device(handle_t handle, int layer_cnt, char *pFileColorLUT);
```

功能描述

初始化 OSD 设备，设置图层数量和颜色查找表（Color LUT）。在打开设备后、创建图层前调用。

参数

- handle：设备句柄
- layer_cnt：图层数量（1-8，最大为8）
- pFileColorLUT：Color LUT 数据指针，指向 COLORLUT_S 结构体

返回值

- 成功：0
- 失败：-1

注意事项

- 最大layer_cnt <= 8（LAYER_HANDLE_MAX）
- LUT 格式：前 8 字节头部信息，后续为 RGB（每 3 字节一组），**OSD设备全局只有一个颜色查找表，为所有图层共用**
- 初始化后设备配置为阻塞模式

错误处理

- layer_cnt > 8 返回 -1

---

osd_get_status

函数原型

```
int osd_get_status(handle_t handle, unsigned char *status);
```

功能描述

获取 OSD 设备当前状态。

参数

- handle：设备句柄
- status：输出参数，用于接收状态值

返回值

- 成功：0
- 失败：-1

3.2 图层管理

osd_create_layer

函数原型

```
int osd_create_layer(handle_t handle, LAYER_HANDLE layer_index, LAYER_ATTR_S *p
```

功能描述

创建新的 OSD 图层。图层是 OSD 显示基本单位，可独立配置管理。

参数

- handle：设备句柄
- layer_index：图层索引（LAYER0_HANDLE ~ LAYER7_HANDLE）
- pstLayer：图层属性结构体指针

返回值

- 成功：0
- 失败：-1

使用示例

```
LAYER_ATTR_S layer_attr;
memset(&layer_attr, 0, sizeof(LAYER_ATTR_S));

layer_attr.codeTYPE = SS_TYPE_QUADRANGLE;

layer_attr.layerStart.layer_start_x = 0;
layer_attr.layerStart.layer_start_y = 0;

layer_attr.layerSize.layer_width = 1920;
layer_attr.layerSize.layer_height = 1080;

layer_attr.layer_rgn.enType = TYPE_GRAPHIC;
layer_attr.layer_rgn.size_s.w = 1920;
layer_attr.layer_rgn.size_s.h = 1080;

int ret = osd_create_layer(handle, LAYER0_HANDLE, &layer_attr);
if (ret < 0) {
    printf("Failed to create layer\n");
}
```

注意事项

- 同一图层索引只能创建一次，重复创建会失败
- 最大 8 个图层
- pstLayer 必须完整初始化
- 创建后**默认禁用**，需 osd_enable_layer 使能，flush功能的API例如osd_flush_quard_rangle也会使能图层
- **坐标原点为图像中心**，范围 [-32768, 32767]

图层结构体说明：LAYER_ATTR_S

```
typedef struct ssLAYER_ATTR_S {
    LAYER_TYPE      codeTYPE;       // SS_TYPE_RLE 或 SS_TYPE_QUADRANGLE
    LAYER_COORD     layerStart;     // 起始坐标
    LAYER_SIZE      layerSize;      // 图层大小
    int             sensor_flag;    // 传感器标志
    QUADRANGLE_S    layer_data_QR;  // 四边形编码数据
    RLE_S           layer_data_RLE; // RLE 编码数据
    RGN_ATTR_S      layer_rgn;      // 区域属性
} LAYER_ATTR_S;
```

错误处理

- 索引已存在：-1
- 超出最大数量：-1

---

osd_destroy_layer

函数原型

```
int osd_destroy_layer(handle_t handle, LAYER_HANDLE layer_index);
```

功能描述

销毁指定图层并释放资源。

参数

- handle：设备句柄
- layer_index：图层索引

返回值

- 成功：0
- 失败：-1（图层不存在或已锁定）

注意事项

- 建议销毁前先 osd_clean_layer
- 锁定图层也可销毁（会从锁定映射移除）
- 销毁后索引可复用

---

osd_enable_layer

函数原型

```
int osd_enable_layer(handle_t handle, LAYER_HANDLE layer_index, bool enable);
```

功能描述

使能/禁用指定图层。使能后才会显示。

参数

- handle：设备句柄
- layer_index：图层索引
- enable：true 使能；false 禁用

返回值

- 成功：0
- 失败：-1

使用示例

```
osd_enable_layer(handle, LAYER0_HANDLE, true);
osd_enable_layer(handle, LAYER0_HANDLE, false);
```

注意事项

- 创建后默认禁用
- 禁用不会清空内容，仅不显示
- 可随时切换状态

---

osd_lock_layer

函数原型

```
int osd_lock_layer(handle_t handle, LAYER_HANDLE layer_index, bool lock);
```

功能描述

锁定/解锁图层。锁定后不能修改，只能查看或销毁。

参数

- handle：设备句柄
- layer_index：图层索引
- lock：true 锁定；false 解锁

返回值

- 成功：0
- 失败：-1（图层不存在）

使用示例

```
osd_lock_layer(handle, LAYER0_HANDLE, true);
osd_lock_layer(handle, LAYER0_HANDLE, false);
```

注意事项

- 锁定图层不能被 osd_clean_layer 清理
- 锁定图层仍可 enable/disable

- 锁定图层可被 destroy

---

osd_clean_layer

函数原型

```
int osd_clean_layer(handle_t handle, LAYER_HANDLE layer_index);
```

功能描述

清空图层内容但**保留图层本身。**

参数

- handle：设备句柄
- layer_index：图层索引

返回值

- 成功：0
- 失败：-1（图层不存在或已锁定）

注意事项

- 仅可清空未锁定图层
- 清空会刷新空数据并重新使能图层（显示空白）

---

osd_clean_all_layer

函数原型

```
int osd_clean_all_layer(handle_t handle);
```

功能描述

清空所有未锁定图层内容。

参数

- handle：设备句柄

返回值

- 成功：0
- 失败：-1（无效句柄）

注意事项

- 仅清理未锁定图层
- 锁定图层不受影响

3.3 DMA 缓冲区管理

osd_alloc_buffer

函数原型

```
int osd_alloc_buffer(handle_t handle, void * &buffer_handle, int buf_size);
```

功能描述

分配 DMA 缓冲区，用于存储编码数据。

参数

- handle：设备句柄
- buffer_handle：返回缓冲区句柄
- buf_size：缓冲区大小（字节）

返回值

- 成功：0，buffer_handle 有效
- 失败：-1，buffer_handle == nullptr

使用示例

```
void *buffer_handle = nullptr;
int buf_size = 1024 * 1024;

int ret = osd_alloc_buffer(handle, buffer_handle, buf_size);
if (ret < 0 || buffer_handle == nullptr) {
    printf("Failed to alloc buffer\n");
    return -1;
}
```

注意事项

- 缓冲区可被多个图层复用
- 分配缓冲区后可以等待，例如使用sleep(0.25)确保硬件稳定

大小建议

- 四边形编码：1024
- RLE 位图编码：0x20000（与图像大小/复杂度相关，若图像显示一部分则证明bufsize不够）

---

osd_delete_buffer

函数原型

```
int osd_delete_buffer(handle_t handle, void * buffer_handle);
```

功能描述

释放已分配 DMA 缓冲区。

参数

- handle：设备句柄

- buffer_handle：缓冲区句柄

返回值

- 成功：0
- 失败：-1

使用示例

```
int ret = osd_delete_buffer(handle, buffer_handle);
if (ret < 0) {
    printf("Failed to delete buffer\n");
}
buffer_handle = nullptr;
```

注意事项

- 释放前确保没有图层正在使用该缓冲区
- 释放后句柄失效

---

osd_get_buffer_ptr

函数原型

```
void *osd_get_buffer_ptr(handle_t handle, void * buffer_handle);
```

功能描述

获取 DMA 缓冲区的虚拟地址指针，可直接访问缓冲区内容。

参数

- handle：设备句柄
- buffer_handle：缓冲区句柄

返回值

- 成功：虚拟地址指针
- 失败：nullptr

使用示例

```
void *ptr = osd_get_buffer_ptr(handle, buffer_handle);
if (ptr == nullptr)
    printf("Failed to get buffer pointer\n");
    return -1;
}
memcpy(ptr, data, size);
```

注意事项

- 指针有效期：缓冲区释放前
- 不要越界写入

---

osd_get_buffer_fd

函数原型

```
int osd_get_buffer_fd(handle_t handle, void * buffer_handle);
```

功能描述

获取 DMA 缓冲区文件描述符（fd），用于 DMA 相关操作。

参数

- handle：设备句柄
- buffer_handle：缓冲区句柄

返回值

- 成功：fd（>0）
- 失败：-1

注意事项

- 不要手动 close 返回的 fd
- 缓冲区释放后 fd 自动失效

---

osd_set_layer_buffer

函数原型

```
int osd_set_layer_buffer(handle_t handle, LAYER_HANDLE layer_index, DMA_BUFFEF
```

功能描述

为指定图层设置 DMA 缓冲区（存储编码数据）。

参数

- handle：设备句柄
- layer_index：图层索引
- dma：DMA 缓冲区属性

返回值

- 成功：0
- 失败：-1（图层不存在）

使用示例

```
DMA_BUFFER_ATTR_S dma_attr;
dma_attr.dma = buffer_handle1;
dma_attr.dma_2 = buffer_handle2;
int ret = osd_set_layer_buffer(handle, LAYER0_HANDLE, dma_attr);
if (ret < 0) {
    printf("Failed to set layer buffer\n");
}
```

注意事项

- dma.dma 必填
- dma.dma_2 可选（双缓冲）
- 单缓冲：仅设置 dma.dma
- 双缓冲：同时设置 dma.dma 与 dma.dma_2

结构体：DMA_BUFFER_ATTR_S

```
typedef struct ssDMA_BUFFER_ATTR_S {
    void* dma;      // 主缓冲区
    void* dma_2;    // 副缓冲区（可选）
} DMA_BUFFER_ATTR_S;
```

3.4 图形操作（四边形）

osd_add_quad_rangle

函数原型

```
int osd_add_quad_rangle(handle_t handle, COVER_ATTR_S *attr);
```

功能描述

向任意匹配的图形图层（TYPE_GRAPHIC）添加四边形（自动查找图层）。

参数

- handle：设备句柄
- attr：四边形属性

返回值

- 成功：0
- 失败：-1（未找到可用图层或添加失败）

使用示例

```
COVER_ATTR_S cover;
cover.colorIdx = 5;
cover.eSolid = TYPE_SOLID;
cover.alpha = TYPE_ALPHA100;

cover.vertex_out.points[0] = {100, 100};
cover.vertex_out.points[1] = {300, 100};
cover.vertex_out.points[2] = {300, 200};
cover.vertex_out.points[3] = {100, 200};

int ret = osd_add_quad_rangle(handle, &cover);
if (ret < 0) {
    printf("Failed to add quadrangle\n");
}
```

注意事项

- 遍历所有未锁定图层，查找 enType == TYPE_GRAPHIC
- 若某图层添加失败会继续尝试下一个图层
- 仅当成功添加到某个图层才返回 0
- 全部失败返回 -1

结构体：COVER_ATTR_S

```
typedef struct tagCOVER_ATTR_S {
    int  colorIdx;            // 颜色索引（0-31）
    QUADRANGLETYPE  eSolid;     // TYPE_SOLID 或 TYPE_HOLLOW
    ALPHATYPE       alpha;      // TYPE_ALPHA25/50/75/100
    VERTEXS_S       vertex_out;  // 外四边形（4点）
    VERTEXS_S       vertex_in;   // 内四边形（空心时有效）
} COVER_ATTR_S;
```

四边形限制

- 必须为**凸四边形**
- 在图层内绘制时，四边形的顶点坐标的单位是像素，坐标原点为图层**左上角**，向右为正，**向下为正**
- **顶点顺序按照顺时针或者逆时针顺序**
- 同一行最多 4 个四边形

- 每层存在最大数量限制

---

osd_flush_quad_rangle

函数原型

```
int osd_flush_quad_rangle(handle_t handle);
```

功能描述

刷新所有图形图层（TYPE_GRAPHIC）的四边形数据到设备。

参数

- handle：设备句柄

返回值

- 成功：0
- 失败：-1

使用示例

```
osd_add_quad_rangle(handle, &cover1);
osd_add_quad_rangle(handle, &cover2);

int ret = osd_flush_quad_rangle(handle);
if (ret < 0) {
    printf("Failed to flush quadrangle\n");
}
```

注意事项

- 仅刷新 TYPE_GRAPHIC 图层
- 刷新包含：编码 → 拷贝 DMA → 更新寄存器 → **使能图层**

---

osd_add_quad_rangle_layer

函数原型

```
int osd_add_quad_rangle_layer(handle_t handle, LAYER_HANDLE layer_index, COVE
```

功能描述

向指定图层添加四边形（指定图层版本）。

参数

- handle：设备句柄
- layer_index：目标图层

- attr：四边形属性

返回值

- 成功：0
- 失败：-1（图层不存在或类型不匹配）

使用示例

```
int ret = osd_add_quad_rangle_layer(handle, LAYER0_HANDLE, &cover);
if (ret < 0) {
    printf("Failed to add quadrangle to layer\n");
}
```

注意事项

- 目标图层必须为 TYPE_GRAPHIC
- 添加后需调用 osd_flush_quad_rangle_layer

---

osd_flush_quad_rangle_layer

函数原型

```
int osd_flush_quad_rangle_layer(handle_t handle, LAYER_HANDLE layer_index);
```

功能描述

刷新指定图层的四边形数据到设备。

参数

- handle：设备句柄
- layer_index：目标图层

返回值

- 成功：0
- 失败：-1（图层不存在或类型不匹配）

使用示例

```
osd_add_quad_rangle_layer(handle, LAYER0_HANDLE, &cover1);
osd_add_quad_rangle_layer(handle, LAYER0_HANDLE, &cover2);

int ret = osd_flush_quad_rangle_layer(handle, LAYER0_HANDLE);
if (ret < 0) {
    printf("Failed to flush layer\n");
}
```

注意事项

- 仅刷新 TYPE_GRAPHIC 图层
- 刷新后**自动使能图层**
- 原子刷新，保证一致性

3.5 位图操作

osd_add_texture

函数原型

```
int osd_add_texture(handle_t handle, BITMAP_INFO_S *info);
```

功能描述

向任意匹配的图像图层（TYPE_IMAGE）添加位图（自动查找图层），位图文件必须为 **.ssbmp**。

参数

- handle：设备句柄
- info：位图信息

返回值

- 成功：0
- 失败：-1（未找到可用图层或添加失败）

使用示例

```
BITMAP_INFO_S bmp_info;
bmp_info.pSSbmpFile = "/path/to/image.ssbmp";
bmp_info.alpha = TYPE_ALPHA100;
bmp_info.position.x = 100;
bmp_info.position.y = 100;

int ret = osd_add_texture(handle, &bmp_info);
if (ret < 0) {
    printf("Failed to add texture\n");
}
```

注意事项

- 遍历未锁定图层，查找 enType == TYPE_IMAGE
- 位图文件必须为 **.ssbmp**
- **位置以左上角坐标为基准定位，超出区域部分会被裁剪**
- 全部失败返回 -1

结构体：BITMAP_INFO_S

```
typedef struct tagBITMAP_INFO_S {
    const char*  pSSbmpFile;  // SSBMP 文件路径
    ALPHATYPE    alpha;       // TYPE_ALPHA25/50/75/100
    POINT_S      position;    // 左上角坐标
} BITMAP_INFO_S;
```

SSBMP 文件格式

- Magic：4 字节 0x5353424D（"SSBM"）
- width：4 字节
- height：4 字节
- colorNum：4 字节
- data：索引颜色数据

---

osd_flush_texture

函数原型

```
int osd_flush_texture(handle_t handle);
```

功能描述

刷新所有图像图层（TYPE_IMAGE）的位图数据到设备。

参数

- handle：设备句柄

返回值

- 成功：0
- 失败：-1

使用示例

```
osd_add_texture(handle, &bmp1);
osd_add_texture(handle, &bmp2);

int ret = osd_flush_texture(handle);
if (ret < 0) {
    printf("Failed to flush texture\n");
}
```

注意事项

- 仅刷新 TYPE_IMAGE 图层
- 刷新包含：RLE 编码 → 拷贝 DMA → 更新寄存器 → **使能图层**

---

osd_add_texture_layer

函数原型

```
int osd_add_texture_layer(handle_t handle, LAYER_HANDLE layer_index, BITMAP_IN
```

功能描述

向指定图层添加位图（指定图层版本）。

参数

- handle：设备句柄
- layer_index：目标图层
- info：位图信息

返回值

- 成功：0
- 失败：-1（图层不存在或类型不匹配）
- 失败：-2（添加失败，如编码数据过大或文件无效）

使用示例

```
BITMAP_INFO_S bmp_info;
bmp_info.pSSbmpFile = "/path/to/image.ssbmp";
bmp_info.alpha = TYPE_ALPHA100;
bmp_info.position.x = 50;
bmp_info.position.y = 50;

int ret = osd_add_texture_layer(handle, LAYER0_HANDLE, &bmp_info);
if (ret < 0) {
    printf("Failed to add texture to layer\n");
}
```

错误码说明

- -1：图层不存在或类型不匹配
- -2：位图添加失败（数据过大/文件无效等）

注意事项

- 目标图层必须为 TYPE_IMAGE
- 添加后需调用 osd_flush_texture_layer

osd_flush_texture_layer

函数原型

```
int osd_flush_texture_layer(handle_t handle, LAYER_HANDLE layer_index);
```

功能描述

刷新指定图层位图数据到设备。

参数

- handle：设备句柄
- layer_index：目标图层

返回值

- 成功：0
- 失败：-1（图层不存在或类型不匹配）

使用示例

```
osd_add_texture_layer(handle, LAYER0_HANDLE, &bmp1);
osd_add_texture_layer(handle, LAYER0_HANDLE, &bmp2);

int ret = osd_flush_texture_layer(handle, LAYER0_HANDLE);
if (ret < 0) {
    printf("Failed to flush texture layer\n");
}
```

注意事项

- 仅刷新 TYPE_IMAGE 图层
- 刷新后自动**使能图层**
- 原子刷新，保证一致性

位图格式转换

位图的输入支持格式为特殊格式ssbmp，提供的python脚本可以支持PNG格式和BMP格式的图像转为ssbmp格式和对应的颜色查找表便于同学们开发。

使用说明：

在bmpimput内准备好需要转换的图片，运行bmp2ssbm.py即可在ssbmpoutput输出ssbmp文件和对应的颜色查找表。

需要安装python环境，同时需要安装pillow（pip install Pillow）。

1. 图片格式转换

- 支持输入格式：BMP（24位/32位）、PNG（RGBA）
- 输入图片位置：文件夹**bmpInput**

思特威logo-彩.
png

- 输出格式：SSBMP（索引图像）+ SSCL（颜色查找表）
- 输出产物位置：文件夹**ssbmpOutput**

思特威logo-彩.ssbmp
思特威logo-彩_colorLUT.sscl

- 透明度处理：**PNG的透明像素会被标记为特殊索引（0x1f）**

2. 单文件模式
- 每个文件独立生成颜色查找表
- 适用于文件之间颜色差异较大的场景

3. 多文件共用LUT模式
- 自动启用：当检测到多个文件时，自动使用共用LUT模式
- 仅支持多个PNG或多个BMP文件
- **共用颜色表：所有文件共用一张颜色查找表（最多30种颜色）**

4. 尺寸压缩（Resize）
- 单个值模式：DEFAULT_RESIZE_WIDTH = 700，所有文件使用相同尺寸
- 数组模式：DEFAULT_RESIZE_WIDTH = [(700, None), (800, 400)]，每个文件使用对应配置
- 等比压缩：高度设为None时**自动保持宽高比**

```
# 默认压缩尺寸（支持数组）单位：像素
DEFAULT_RESIZE_WIDTH = 700  # 或 [(700, None), (800, 400)]
DEFAULT_RESIZE_HEIGHT = None
```

5. 颜色压缩
- 自动触发：当颜色数超过**30**时自动进行颜色减少，**颜色压缩后，画面会有一定程度的损失**

- 智能选择：按颜色使用频率和色系分类选择保留颜色
-Terminal会输出最终选择的颜色的RGB，YUV值和他的颜色类别

```
# 最大颜色数，可以设置小于30的数字
COLORLUT_MAX = 30
```

```
Problems   Output   Debug Console   Terminal   Ports
颜色列表 (共30种):
索引     RGB            YUV            色类
-----------------------------------------------
0      ( 10,122,192)    ( 96,182, 66)    蓝绿
1      (  8,119,190)    ( 94,182, 66)    蓝绿
2      (  0,  0,  0)    (  0,128,128)    黑色
3      (113,112,113)    (112,128,128)    灰色
4      ( 19,135,202)    (108,181, 64)    蓝绿
5      ( 17,132,200)    (105,181, 65)    蓝绿
6      ( 15,130,198)    (103,181, 65)    蓝绿
7      ( 14,128,196)    (102,181, 65)    蓝绿
8      ( 12,125,194)    ( 99,181, 65)    蓝绿
9      (  7,116,187)    ( 91,182, 68)    蓝绿
10     ( 26,147,211)    (118,180, 62)    蓝绿
11     ( 28,150,213)    (121,179, 61)    蓝绿
12     ( 24,138,204)    (111,180, 65)    蓝绿
13     ( 22,140,206)    (112,181, 63)    蓝绿
14     ( 23,143,208)    (115,180, 62)    蓝绿
15     ( 25,145,209)    (116,180, 63)    蓝绿
16     ( 20,138,204)    (110,181, 63)    蓝绿
17     ( 30,146,210)    (119,179, 64)    蓝绿
18     ( 32,151,212)    (122,178, 63)    蓝绿
19     ( 31,149,211)    (121,178, 63)    蓝绿
20     ( 34,152,214)    (124,178, 63)    蓝绿
21     ( 35,154,215)    (125,178, 63)    蓝绿
22     ( 37,156,217)    (127,178, 63)    蓝绿
23     ( 38,158,218)    (129,178, 63)    蓝绿
24     ( 40,160,220)    (131,178, 63)    蓝绿
25     ( 41,162,221)    (133,177, 62)    蓝绿
26     (  4,112,184)    ( 88,182, 68)    蓝绿
27     ( 43,164,223)    (135,177, 62)    蓝绿
28     (  5,113,186)    ( 89,182, 68)    蓝绿
29     ( 36,136,193)    (113,173, 73)    蓝绿
```

6. 配对编码

- **最终A1输出格式为YUV格式**，相邻的像素共用U和V。若一组像素颜色不同会导致颜色异常，如下图所示，本脚本通过默认开启 ENABLE_PAIR_ENCODING 将**一组像素使用同一颜色**，保证颜色正常，但会**损失一定的平滑度**，可以选择关闭
- 同时，选择位置的时候选择**偶数起点，例如（0，0）**
- 默认启用：两像素一组编码，相邻两个像素使用相同的颜色索引
- 智能选择：优先保留非透明像素的颜色，透明像素优先被替换

```
# 是否启用配对编码
ENABLE_PAIR_ENCODING = True
```

7. 解码debug功能

- 自动解码：转换完成后自动解码SSBMP回BMP用于验证debug，省去了烧录验证的繁琐
- 透明像素显示：原透明像素在解码后的BMP中显示为黄色便于验证debug

当多图输入时，由于**共用最大30色的颜色查找表**，对于每一张图片，画面和颜色空间会有更大的损失

坐标系统说明

- 在图层内绘制图形或位图时，以所在图层的**左上角**为坐标原点
- X：向右为正
- Y：**向下为正**
- 单位：像素

颜色索引说明

- 范围：0-31（32 色：其中30，31是特殊索引，索引30为反色，索引31为透明。实际颜色索引最大支持0-29）
- 颜色值由 **Color LUT** 配置
- 所有位图/图形**共享唯一颜色查找表**
- LUT：每 **3 字节一组**

透明度级别

枚举值	不透明度	说明
TYPE_ALPHA25	25%	较透明
TYPE_ALPHA50	50%	半透明
TYPE_ALPHA75	75%	较不透明
TYPE_ALPHA100	100%	完全不透明